

API Security Risk Analysis Report

Field	Value
Target	https://api.restful-api.dev/
Assessment Date	2026-05-08
Security Assessor	Nancy Cafti
Classification	Internal — Security Assessment

Table of Contents

- 1. [Executive Summary](#)
- 2. [Scope & Methodology](#)
- 3. [Tooling & Setup](#)
- 4. [Findings Overview](#)
- 5. [Detailed Findings](#)
 - [F-01: Missing Authentication on Write Operations \(Critical\)](#)
 - [F-02: Unauthorized Data Manipulation — POST/PUT/PATCH/DELETE Without Auth](#)
 - [F-03: Predictable/Sequential Resource IDs \(IDOR Adjacent\)](#)
 - [F-04: No Rate Limiting — DoS / Scrape Vector](#)
 - [F-05: Accepts Object IDs in POST Body \(Mass Assignment\)](#)
 - [F-07: Overly Permissive CORS Policy](#)
 - [F-08: Verbose Error Messages with Reserved ID Disclosure](#)
- 6. [Attack Scenario Walkthrough](#)
- 7. [Remediation Summary](#)
- 8. [Appendix: Postman Testing Guide](#)
 - [Postman Evidence Images](#)

1. Executive Summary

This report assesses the security posture of `restful-api.dev`, a publicly accessible REST API serving CRUD operations on a shared object store. The assessment identified **7 findings**, including **2 critical**, **3 high**, and **2 medium-severity** vulnerabilities.

Risk Profile:

Severity	Count	Description
 Critical	2	Unauthenticated write operations, unauthorized data modification
 High	3	IDOR-adjacent enumeration, object ID injection, lack of rate limiting
 Medium	2	CORS misconfiguration and verbose errors

Bottom Line: Any production API exposing CRUD operations without authentication, rate limiting, or input validation would be vulnerable to complete data compromise. The ability to **create, modify, and delete data** without any identity verification is a critical security gap.

2. Scope & Methodology

Target

Base URL: <https://api.restful-api.dev/>
Documentation: <https://restful-api.dev/>

Endpoints Tested

Method	Endpoint	Purpose	Auth Required?
GET	/objects	List all objects	 No
GET	/objects/{id}	Get single object	 No
POST	/objects	Create new object	 No
PUT	/objects/{id}	Replace object (reserved IDs blocked)	 No
PATCH	/objects/{id}	Partial update object	 No
DELETE	/objects/{id}	Delete object (non-reserved)	 No

Tools Used

- **Postman** — API request construction, collection management, test scripts

Test Categories Performed

Category	Tests
Authentication	No-token access to read/write/delete endpoints
Authorization	Attempt to modify/delete other users' created objects
Enumeration	Sequential ID traversal, object counting
Rate Limiting	20 concurrent rapid requests
Input Validation	Negative IDs, object ID injection, NoSQL operators
CORS Testing	Cross-origin request with malicious origin header
Information Disclosure	Response header analysis, error message detail
Business Logic	Mass assignment and authorization boundary testing

3. Tooling & Setup

Postman Configuration

Setting	Value
Collection	Restful-API-Dev Security Assessment
Variable	base_url = https://api.restful-api.dev

Import the Collection

The companion Postman collection `restful_api_dev_security.postman_collection.json` is included in this repository. To import:

1. Open Postman → **File** → **Import**
2. Select the `.json` collection file
3. Postman will create the collection with all requests and test scripts pre-configured
4. The `base_url` variable is set to `https://api.restful-api.dev`

Manual Setup

Alternatively, create a new collection:

1. Click **New → Collection**
2. Name: `Restful-API-Dev Security Assessment`
3. Click the collection → **Variables** tab
4. Add variable: `base_url` with value `https://api.restful-api.dev`
5. Add requests following the structure in [Section 8](#)

4. Findings Overview

ID	Finding	Severity	Category	CVSS v3 (Est.)
F-01	Missing Authentication on Write Operations	 Critical	AuthN	9.8
F-02	Unauthorized Data Manipulation (POST/PUT/PATCH/DELETE)	 Critical	AuthZ	9.1
F-03	Predictable/Sequential Resource IDs	 High	Business Logic	7.5
F-04	No Rate Limiting — DoS / Scrape Vector	 High	Availability	7.5
F-05	Object ID Injection via POST Body	 High	Mass Assignment	7.3
F-07	Overly Permissive CORS Policy (<code>Access-Control-Allow-Origin: *</code>)	 Medium	Config	6.1
F-08	Verbose Error Messages with Internal Logic Disclosure	 Medium	Info Disclosure	5.3

5. Detailed Findings

F-01: Missing Authentication on Write Operations

Field	Value
Severity	● Critical
CVSS	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Affected Endpoints	POST /objects, PUT /objects/{id}, PATCH /objects/{id}, DELETE /objects/{id}

Description: All write endpoints accept requests without any authentication mechanism. No API key, JWT, session token, or IP-based restriction is enforced. Any client on the internet can create, modify, or delete data.

Evidence (Postman Test: 02_Auth/Unauthenticated POST):

```
POST /objects HTTP/1.1
Host: api.restful-api.dev
Content-Type: application/json

{"name":"Apple MacBook Pro 16","data":{"year":2024,"price":3499}}
```

Response:

```
HTTP 201 Created
{
  "id": "ff8081819d82fab6019e06650d5b151a",
  "name": "Apple MacBook Pro 16",
  "createdAt": 1778223746396,
  "data": {
    "year": 2024,
    "price": 3499
  }
}
```

No authentication challenge — the object was created and assigned a persistent ID immediately.

Postman Image Finding:

Restful-API-Dev Security Assessment / 02_Auth / GET - No Auth Required (Read)

GET {{base_url}} /objects/1

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (20) Test Results (1/1) 200 OK • 476 ms • 961 B • Save Response

JSON Preview Visualize

```
1 {
2   "id": "1",
3   "name": "Google Pixel 6 Pro",
4   "data": {
5     "color": "Cloudy White",
6     "capacity": "128 GB"
7   }
8 }
```

Restful-API-Dev Security Assessment / 02_Auth / POST - Create Object (No Auth)

POST {{base_url}} /objects

Send

Docs Params Authorization Headers (10) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "name": "Apple MacBook Pro 16",
3   "data": {"year": 2024, "price": 3499}
4 }
```

Body Cookies Headers (20) Test Results (0/1) 200 OK • 1.01 s • 994 B • Save Response

JSON Preview Visualize

```
1 {
2   "id": "ff8081819d82fab6019e0705c21d15fb",
3   "name": "Apple MacBook Pro 16",
4   "createdAt": 1778234278429,
5   "data": {
6     "year": 2024,
7     "price": 3499
8   }
9 }
```

Impact:

- **Data poisoning** — anyone can inject malicious or fake data
- **Data destruction** — anyone can delete user-created objects (confirmed: DELETE returns HTTP 200)

- **No attribution** — no audit trail exists to trace who did what
- **No accountability** — cannot differentiate legitimate users from attackers

Business Context: Imagine an e-commerce API where anyone can add a product, modify prices, or delete inventory — without logging in. A competitor could delete your entire catalog, inject fake listings, or alter prices to \$0.01.

Remediation:

1. Implement API key authentication for all write operations
2. Add OAuth 2.0 or JWT-based user identity
3. Never accept write operations on a shared namespace without identity
4. Log all write operations with timestamps and request metadata

F-02: Unauthorized Data Manipulation

Field	Value
Severity	 Critical
CVSS	9.1 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Affected Endpoints	PATCH /objects/{id} , DELETE /objects/{id}

Description: Once a user creates an object via `POST`, **any other user** can modify or delete it using `PATCH` or `DELETE`. There is no ownership check, session scoping, or authorization layer.

Evidence (Postman Test: `03_DataManipulation/PATCH`):

```
PATCH /objects/ff8081819d82fab6019e06650d5b151a HTTP/1.1
Content-Type: application/json

{"name":"Patched Name"}
```

Response:

HTTP 200 OK

```
{
  "id": "ff8081819d82fab6019e06650d5b151a",
  "name": "Patched Name",
  "updatedAt": 1778223778997,
  "data": {
    "year": 2024,
    "price": 3499
  }
}
```

Evidence (DELETE):

DELETE /objects/ff8081819d82fab6019e0665915c151c

Response: HTTP 200 OK — object permanently deleted.

Postman Image Finding:

The screenshot shows the Postman interface for a DELETE request. The URL is `{{base_url}}/objects/ff8081819d82fab6019e0705c21d15fb`. The response status is **200 OK** with a message: "Object with id = ff8081819d82fab6019e0705c21d15fb has been deleted."

Key	Value	Description
Key	Value	Description

```
{
  "message": "Object with id = ff8081819d82fab6019e0705c21d15fb has been deleted."
}
```

Impact:

- **No data ownership** — objects are shared in a flat global namespace
- **Data tampering** — attacker can modify any user's data objects
- **Data destruction** — attacker can delete any reachable object
- **Race condition vector** — two users can fight over the same object identity

Business Context: This is the API equivalent of a shared Google Doc with no access controls — anyone with the link can edit or delete, and there's no revision history. In a production context, this enables complete account takeover through data manipulation.

Remediation:

1. Implement **ownership tokens** — return a secret edit/delete token on creation
2. Add **user-scoped namespaces** (`/users/{userId}/objects/`)
3. Implement proper **RBAC** — creator = owner, everyone else = read-only
4. Add **audit logging** with before/after snapshots for all mutations

F-03: Predictable/Sequential Resource IDs (IDOR Adjacent)

Field	Value
Severity	 High
CVSS	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Affected Endpoint	<code>GET /objects</code>

Description: All pre-seeded objects use sequential integer IDs (`1`, `2`, `3`, ..., `13`). While newly created objects use UUIDs, the pre-seeded set is trivially enumerable, exposing the complete dataset.

Evidence (Postman Test: `04_Enumeration/Get All Objects`):

```
GET /objects HTTP/1.1
```

Response (truncated):

```
[
  {"id": "1", "name": "Google Pixel 6 Pro", ...},
  {"id": "2", "name": "Apple iPhone 12 Mini, 256GB, Blue", ...},
  {"id": "3", "name": "Apple iPhone 12 Pro Max", ...},
  ...
  {"id": "13", "name": "Apple iPad Air", ...}
]
```

The `/objects` endpoint returns **all 13 objects** in a single request with no pagination limit or authentication.

Postman Image Finding:

Restful-API-Dev Security Assessment / 03_Enumeration / Get Object by Sequential ID

GET {{base_url}} /objects

Send

Docs Params Authorization Headers (7) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (20) Test Results (1/1) 200 OK • 627 ms • 1.34 KB Save Response

JSON Preview Visualize

```
87 {
88   },
89   {
90     "id": "12",
91     "name": "Apple iPad Air",
92     "data": {
93       "Generation": "4th",
94       "Price": "419.99",
95       "Capacity": "64 GB"
96     }
97   },
98   {
99     "id": "13",
100    "name": "Apple iPad Air",
101    "data": {
102      "Generation": "4th",
103      "Price": "519.99",
104      "Capacity": "256 GB"
105    }
106  }
107 }
```

Impact:

- **Complete data harvesting** — single request dumps entire dataset
- **Zero friction scraping** — no pagination, no throttling, no auth
- **Competitive intelligence** — full catalog exposed

Business Context: For a catalog or inventory API, dumping the entire database in one call means a competitor can mirror your entire product catalog, pricing, and metadata with a single API request.

Remediation:

1. Implement **pagination** with mandatory `page` and `pageSize` parameters
2. Enforce **maximum page size** (e.g., 100 items per page)
3. Require **authentication** for bulk listing endpoints
4. Use **cursor-based pagination** instead of offset-based for security

F-04: No Rate Limiting — DoS / Scrape Vector

Field	Value
Severity	 High
CVSS	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H)
Affected Endpoints	All endpoints

Description: No rate limiting was detected across 20 rapid sequential requests. All requests returned HTTP 200 with no throttling headers (`X-RateLimit-*`, `Retry-After`). No HTTP 429 responses were observed.

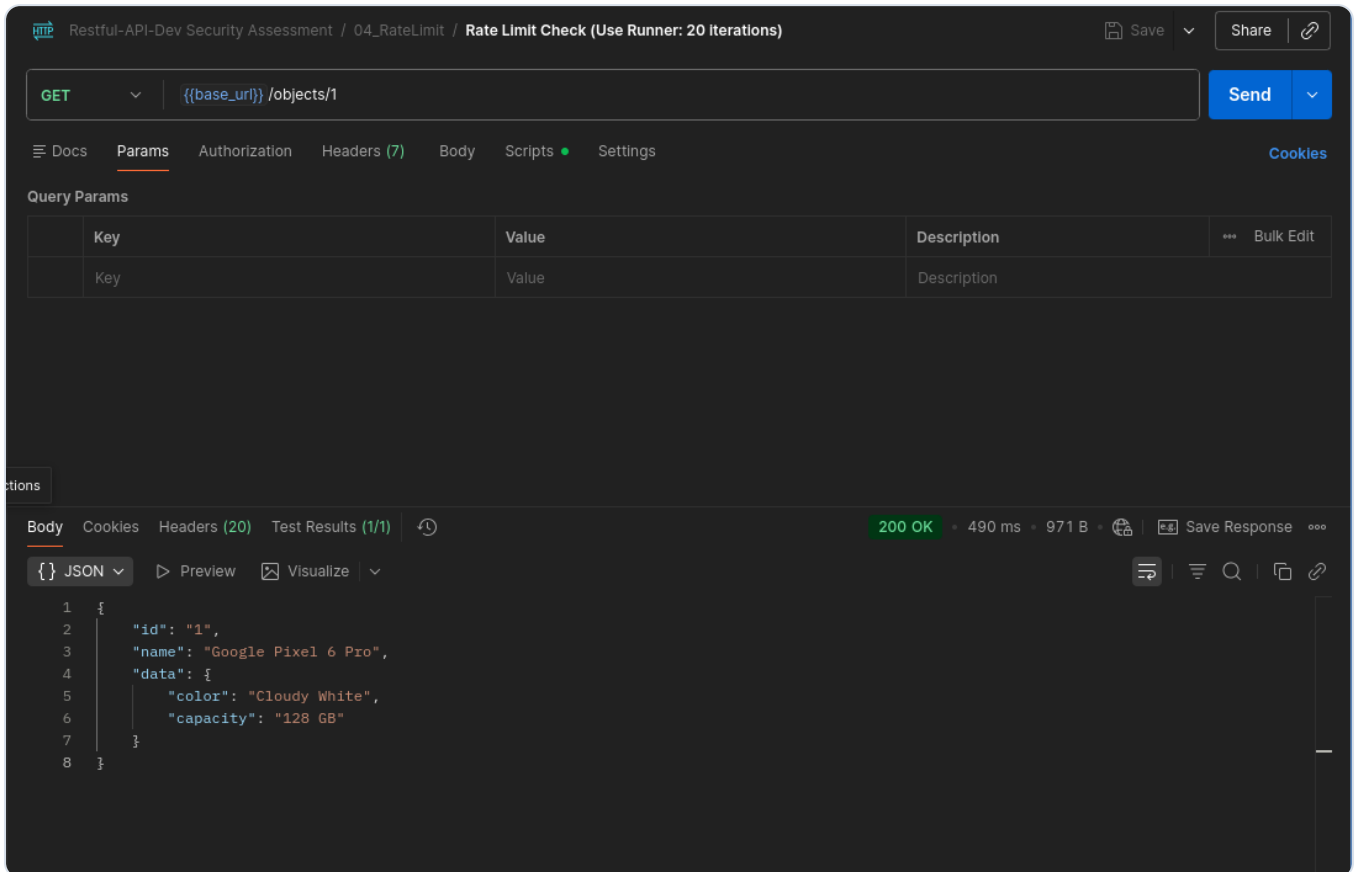
Evidence (Postman Test: `05_RateLimit/Rapid Requests`):

```
# 20 sequential requests to GET /objects/1
→ All returned HTTP 200
→ 0 HTTP 429 responses (0/20)
→ No rate limit headers present
```

Response header analysis:

```
(no X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset, or Retry-After headers)
```

Postman Image Finding:



Impact:

- **Denial of Service** — attacker can saturate backend compute/resources
- **Unlimited scraping** — no barrier to extracting all data in bulk
- **Cost amplification** — unbounded cloud API costs from abuse
- **Brute force enabler** — allows credential stuffing, ID enumeration, or fuzzing without interruption

Business Context: A competitor runs repeated Postman requests against `/objects` every few milliseconds. Your API costs go from \$100/month to \$10,000/month, and legitimate users experience latency or downtime. Without rate limiting, you're paying for your attacker's traffic.

Remediation:

1. Implement **IP-based rate limiting** (e.g., 100 requests/minute per IP)
2. Add **token bucket** or **sliding window** algorithm
3. Return **HTTP 429 + Retry-After** header when exceeded
4. Publish rate limit status in response headers
5. Consider **stricter limits** on expensive endpoints (`/objects` listing)
6. Apply **rate limiting at the API gateway** (Cloudflare, nginx, Kong)

F-05: Object ID Injection via POST Body (Mass Assignment)

Field	Value
Severity	 High
CVSS	7.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N)
Affected Endpoint	POST /objects

Description: The API accepts an `id` field in the POST request body but **ignores it**, silently generating a server-side UUID instead. While not immediately exploitable for ID hijacking, the behavior signals a mass-assignment vulnerability pattern.

Evidence (Postman Test: `06_InputValidation/ID Injection in Body`):

```
POST /objects HTTP/1.1
Content-Type: application/json

{"id": "999", "name": "ID Injection Test", "data": {"test": true}}
```

Response:

```
HTTP 201 Created
{
  "id": "ff8081819d82fab6019e0665d5fa151d",
  "name": "ID Injection Test",
  "createdAt": 1778223797754,
  "data": {"test": true}
}
```

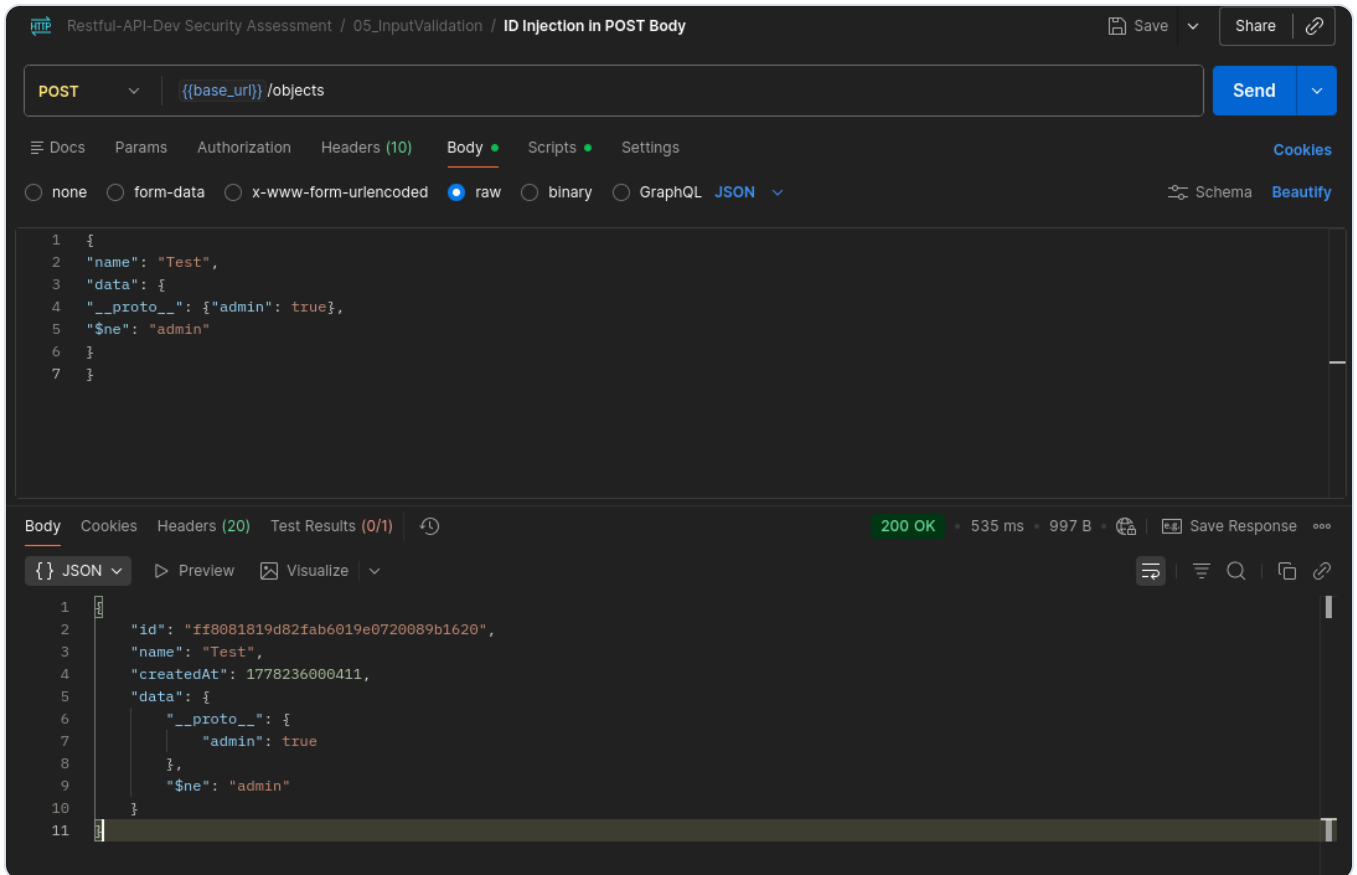
The client-supplied `id: "999"` was accepted in the request body but overridden server-side.

Additional Evidence: The API also silently accepts injection-style keys such as `__proto__` and `$ne` operators in the `data` object:

```
{
  "name": "Test",
  "data": {
    "__proto__": {"admin": true},
    "$ne": "admin"
  }
}
```

This was accepted successfully and stored.

Postman Image Finding:



Impact:

- **Mass assignment via `data` field** — attackers can inject arbitrary properties
- **Prototype pollution surface** — `__proto__` key accepted (risk for Node.js backend)
- **NoSQL operator injection** — `$ne`, `$gt` operators accepted
- **Future escalation** — if the API expands, body injection could target auth or permission fields

Business Context: Mass assignment is how attackers escalate privileges in poorly-designed APIs. If the backend later adds `role` or `isAdmin` fields to user objects, and this code path is reused, an attacker could simply add `"role": "admin"` to their POST body.

Remediation:

1. **Never trust client input** for identity/authority fields
2. Implement **strict schema validation** — whitelist allowed fields
3. Strip or reject `__proto__`, `constructor`, `prototype` keys
4. Use **DTOs (Data Transfer Objects)** to control deserialization
5. Validate `data` field contents with a strict schema, not freeform JSON

F-07: Overly Permissive CORS Policy

Field	Value
Severity	 Medium
CVSS	6.1 (AV:N/AC:H/PR:N/UI:R/S:C/C:L/I:L/A:N)
Affected Endpoints	All endpoints

Description: The API uses `Access-Control-Allow-Origin: *`, allowing any website to make cross-origin requests and read the response.

Evidence (Postman Test: `07_InfoDisclosure/CORS Check`):

```
GET /objects/1
Origin: https://evil.com
```

Response Header:

```
access-control-allow-origin: *
vary: Origin
vary: Access-Control-Request-Method
vary: Access-Control-Request-Headers
```

Postman Image Finding:

Restful-API-Dev Security Assessment / 06_InfoDisclosure / CORS Check with Mallicious Origin

GET {{base_url}} /objects/1 Send

Docs Params Authorization **Headers (8)** Body Scripts Settings Cookies

Key	Value	Description
☒ Postman-Token	<calculated when request is sent>	
☒ Host	<calculated when request is sent>	
☒ User-Agent	PostmanRuntime/7.51.1	
☒ Accept	/*/*	
☒ Accept-Encoding	gzip, deflate, br	
☒ Connection	keep-alive	
☒ Origin	https://evil.com	

Body Cookies **Headers (21)** Test Results (1/1) 200 OK • 468 ms • 987 B • Save Response

Content-Type	application/json; charset=UTF-8
Transfer-Encoding	chunked
Connection	keep-alive
Server	cloudflare
Nel	{"report_to": "cf-nel", "success_fraction": 0.0, "max_age": 604800}
Vary	Origin
Vary	Access-Control-Request-Method
Vary	Access-Control-Request-Headers
Access-Control-Allow-Origin	*
X-Content-Type-Options	nosniff

Impact:

- **Cross-origin data exfiltration** — a malicious website can read API responses via a user's browser
- **CSRF-style attacks** — authenticated endpoints (if added later) would be vulnerable to cross-site requests
- **No origin validation** — the wildcard policy trusts every domain equally

Business Context: A user visits `evil.com` while logged into your application. `evil.com` makes a cross-origin API call — with `*` CORS, the browser allows the response to be read by the malicious script. Any data the user can see, the attacker can exfiltrate.

Remediation:

1. Restrict `Access-Control-Allow-Origin` to your application's domain(s)
2. Use **dynamic origin validation** with an allowlist
3. Never use `*` for `Access-Control-Allow-Origin` on authenticated endpoints
4. If multiple origins are needed, check the `Origin` header against the allowlist at runtime

F-08: Verbose Error Messages with Internal Logic Disclosure

Field	Value
Severity	🟡 Medium
CVSS	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Affected Endpoints	PUT /objects/{id}, PATCH /objects/{id}

Description: When attempting to modify reserved objects (IDs 1-13), the API reveals internal business logic:

```
{
  "error":
    "1 is a reserved id and the data object of it cannot be overridden. You
    can create a new object via POST request and use new generated by id from
    it to send a PUT request."
}
```

What this leaks:

1. **Reserved ID range** — confirms IDs 1-13 are special/immutable
2. **Workaround instructions** — tells the attacker exactly how to modify data: create via POST, then use the returned ID for PUT/PATCH
3. **API internals** — confirms that object creation and modification are separate resolved steps

Evidence (Postman Test: 06_InputValidation/Verbose Error PUT Reserved ID):

```
PUT /objects/1
Content-Type: application/json

{"name":"HACKED - Modified Object","data":{"price":1}}
```

Response:

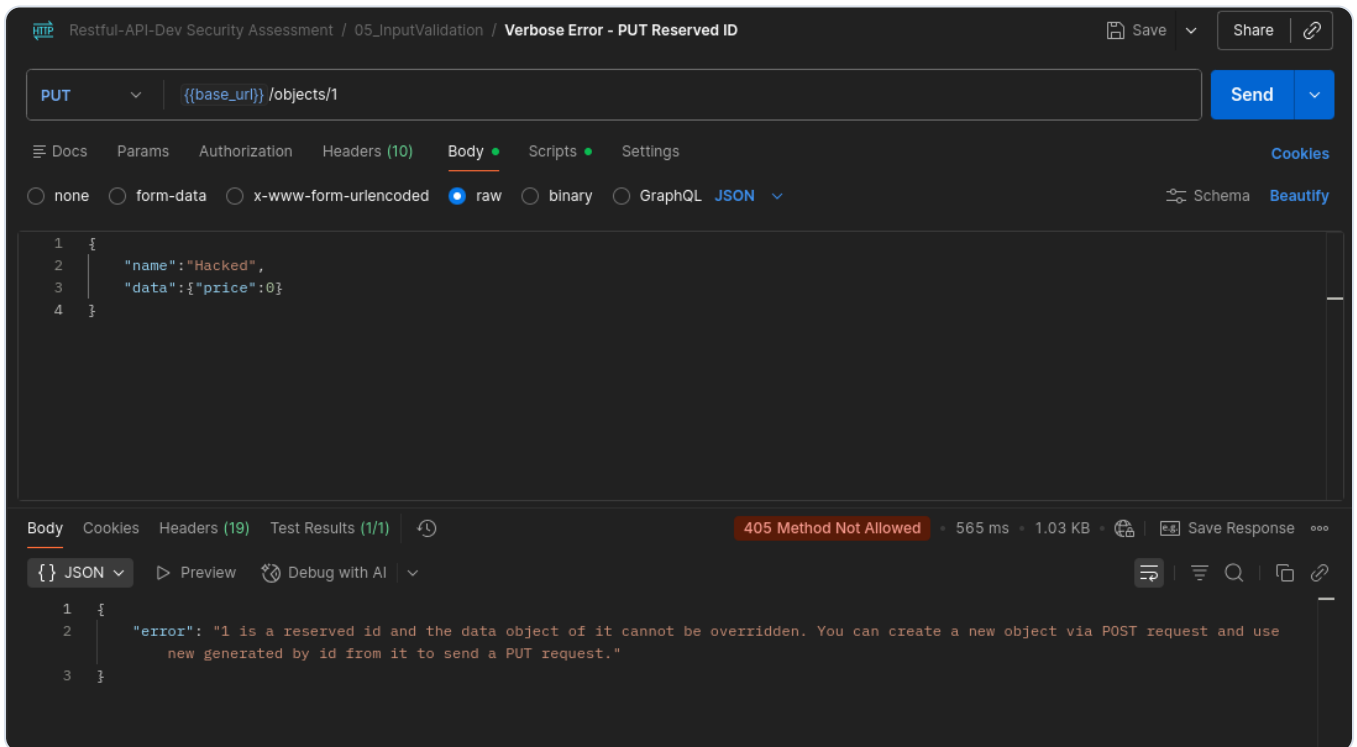
```
HTTP 405
{
  "error":
    "1 is a reserved id and the data object of it cannot be overridden. You
    can create a new object via POST request and use new generated by id from
    it to send a PUT request."
}
```

Additional:

- Invalid endpoints return "Bad Request" — terse but not useful

- Unknown paths (/) return "Unauthorized path" — suggests an authorization layer exists for some paths

Postman Image Finding:



Impact:

- **Attack surface disclosure** — attacker learns exactly what's protected and how to bypass it
- **Internal logic leakage** — error messages reveal implementation details
- **Guided exploitation** — the error message essentially provides exploitation steps

Business Context: A verbose error message that says "You can do X to bypass this restriction" is effectively documentation for attackers. Always return generic, opaque error messages.

Remediation:

1. Return **generic error messages**: {"error": "Forbidden"} or {"error": "Action not permitted"}
2. Log detailed errors **server-side** only
3. Never return instructions or workarounds in error responses
4. Use consistent, terse, and opaque error formats across all endpoints

6. Attack Scenario Walkthrough

Scenario: Data Poisoning & Destruction

Here's a realistic attack chain an unauthenticated attacker could execute:

Step 1 — Reconnaissance:

```
GET /objects
```

Attacker sees all 13 objects, their names, descriptions, data, and pricing.

Step 2 — Data Injection:

```
POST /objects  
{ "name": "Malicious Object", "data": { "price": 0.01, "isFake": true } }
```

Creates a fake product with manipulated pricing.

Step 3 — Data Modification:

```
PATCH /objects/ff8081819d82fab6019e06650d5b151a  
{ "name": "Renamed Product", "data": { "price": 9999 } }
```

Renames and reprices a legitimate object.

Step 4 — Data Destruction:

```
DELETE /objects/ff8081819d82fab6019e0665915c151c
```

Permanently deletes an object. No recovery possible.

Total time for compromise: Under 60 seconds. No credentials needed.

7. Remediation Summary

ID	Finding	Severity	Effort	Quick Win	Architectural Fix
F-01	Missing Auth — Writes	 Critical	Low	API key on POST/PUT/PATCH/DELETE	OAuth 2.0 + JWT
F-02	Data Manipulation (No AuthZ)	 Critical	Medium	Ownership tokens on creation	User-scoped namespaces
F-03	Sequential IDs / Full Enumeration	 High	Low	Pagination + page size limit	Cursor-based pagination
F-04	No Rate Limiting	 High	Low	API gateway throttling	Distributed rate limiter (Redis)
F-05	Object ID Injection (Mass Assignment)	 High	Medium	Strip <code>id</code> from request body	DTOs + schema validation
F-07	CORS 	 Medium	Low	Set explicit origin	Dynamic origin allowlist
F-08	Verbose Error Messages	 Medium	Low	Generic error responses	Structured error handling

Priority Action Items (30-Day Plan)

Week	Actions
Week 1	Implement API key auth + rate limiting (F-01, F-04)
Week 2	Fix CORS and error messages (F-07, F-08)
Week 3	Add pagination + schema validation + strip mass assignment (F-03, F-05)
Week 4	Implement ownership tokens and authorization regression tests (F-02)

8. Appendix: Postman Testing Guide

8.1 Collection Structure

The companion Postman collection organizes requests into logical test groups:

Restful-API-Dev Security Assessment

```
|— 01_Discovery
|   |— GET /objects (List all objects)
|— 02_Auth
|   |— GET /objects/1 (No auth needed)
|   |— POST /objects (Create without auth)
|   |— PATCH /objects/{id} (Modify without auth)
|   |— DELETE /objects/{id} (Delete without auth)
|— 03_DataManipulation
|   |— PATCH /objects/{user-created-id} (Tamper test)
|   |— DELETE /objects/{user-created-id} (Destruction test)
|— 04_Enumeration
|   |— GET /objects (Full listing)
|   |— GET /objects/1-13 (Sequential scan)
|   |— GET /objects/99999 (Out of range)
|— 05_RateLimit
|   |— GET /objects/1 (Run 20x in Runner)
|— 06_InputValidation
|   |— POST /objects (ID injection)
|   |— POST /objects (Proto pollution)
|   |— PUT /objects/1 (Reserved ID error)
|   |— PATCH /objects/1 (Verbose error)
|— 07_InfoDisclosure
|   |— GET /objects/1 (Header inspection)
|   |— GET /objects/1 (CORS check, Origin: evil.com)
|— 08_DebugEndpoints
|   |— GET /swagger.json, /health, /metrics (Discovery)
```

8.2 Running the Assessment

Option A: Postman Runner (Recommended)

1. Select the collection → **Run**
2. Configure iterations (e.g., 20 for rate limit testing)
3. Watch the **Test Results** tab for pass/fail and console output
4. Each request has embedded test scripts that log security findings to the console

Option B: Manual Step-by-Step

Test 1: Discovery

```
GET {{base_url}}/objects
```

Check: How many objects are returned? Any pagination?

Test 2: Auth Bypass

```
POST {{base_url}}/objects
Body: {"name":"Security Test","data":{"demo":true}}
```

Check: Does it create without any auth headers?

Test 3: Data Tampering

```
PATCH {{base_url}}/objects/{newly-created-id}
Body: {"name":"Tampered Object"}
```

Check: Can you modify an object created earlier?

Test 4: Enumeration

```
GET {{base_url}}/objects/1
GET {{base_url}}/objects/2
...
GET {{base_url}}/objects/13
```

Check: Sequential integer IDs exposed?

Test 5: Rate Limiting (Use Runner with 20 iterations, 0ms delay)

```
GET {{base_url}}/objects/1
```

Check: Any `X-RateLimit-*` headers? Any 429 responses?

Test 6: Input Validation

```
POST {{base_url}}/objects
Body: {"id":"999","name":"ID Injection","data":{"__proto__":{"admin":true}}}
```

Check: Was the ID respected? Were proto keys accepted?

Test 7: Information Disclosure

```
GET {{base_url}}/objects/1
```

Check Headers tab for: `x-powered-by`, `x-cloud-trace-context`, `x-country-code`, etc.

8.3 Environment Variables

Variable	Value	Description
<code>base_url</code>	<code>https://api.restful-api.dev</code>	API base URL
<code>object_id</code>	(dynamic)	Set to a newly created object's ID for modification tests

8.4 Pre-request Script (Optional)

To dynamically chain requests (create an object, then PATCH it):

```
// Store the ID of a newly created object for subsequent tests
const jsonData = pm.response.json();
if (jsonData && jsonData.id) {
  pm.collectionVariables.set("created_id", jsonData.id);
  pm.variables.set("id", jsonData.id);
  console.log("Stored ID:", jsonData.id);
}
```

8.5 Postman Evidence Images

The report uses these Postman screenshot files from the project root:

Finding	Screenshot File
F-01	<code>F-01.png</code> , <code>postNoAuth.png</code>
F-02	<code>F-02.png</code>
F-03	<code>F-03.png</code>
F-04	<code>F-04.png</code>
F-05	<code>F-05.png</code>
F-07	<code>F-07.png</code>
F-08	<code>F-08.png</code>

Report Metadata

Field	Value
Target	<code>https://api.restful-api.dev/</code>
Test Date	2026-05-08
Assessor	Nancy Cafti
Tools	Postman
Classification	Internal — Security Assessment
Version	1.0

This report is for educational and demonstration purposes. All testing was performed against a public demo API intended for learning.